1.Methodology

1.1 Machine & Runtime Environment

- **Platform**: Linux (GitHub Codespaces / WSL)
- **CPU**: Document what lscpu reports (architecture, model, cores/threads, L1/L2/L3 sizes)
- **Memory**: From free -h
- **OS & Kernel**: From uname -srmo
- **Compiler**: gcc (version shown by gcc --version) with OpenMP runtime (libgomp)

1.2 OpenMP Configuration (Environment Variables)

We used the following OpenMP environment variables to make thread placement explicit and runs repeatable:

- `OMP_NUM_THREADS=p` — total threads per run
- `OMP_PROC_BIND=spread` — bind threads across cores (minimize contention)
- `OMP_PLACES=cores` — place threads on physical cores
- `OMP_SCHEDULE` — typically unused since we control schedule via CLI; default is shown in `sysinfo.txt`.

1.3 Data Collection & Logging

All experiments stream line-delimited JSON to files, one line per run (machine-parsable).

2.Discussion(Amdahl, Gustafson, K, Scheduling, False sharing mitigation)

Amdahl overlays are most informative when compute dominates; they reveal the apparent serial fraction that caps strong scaling.

Gustafson overlays are best for understanding how well the system absorbs proportionally more work; they still deviate when bandwidth and overhead grow with pp.

K is the pivot knob: larger K drives the workload into a regime where both models align better with measurements.

Scheduling trades overhead for balance; static for uniform work, dynamic/guided for irregular.

False sharing mitigation (padded partials) can help in memory-bound extremes; otherwise, reductions suffice.

3. Accuracy Comparison & Practical Recommendations

For $f(x)$=sin(x)f(x)=sin(x) on [0,π][0,π], our log–log error study confirms the classical accuracy orders:

- **Trapezoid rule:** empirical slope ≈ **2**, matching its $O(h$2)O(h2) error.
- **Simpson's rule:** empirical slope ≈ **4**, matching its $O(h$4)O(h4) error.

Simpson consistently achieved **much smaller error** than trapezoid at the same nn, with roughly **16×** improvement each time hh halves, compared to **4×** for trapezoid. Deviations at extremely fine grids were due to floating-point round-off, not the methods themselves.

Practical recommendations

- **Use Simpson's rule** when:
    - the integrand is **smooth** (like sin(x)sin(x));
    - high accuracy per function evaluation is desired;
    - moderate increases in computational cost are acceptable.
- **Use the trapezoid rule** when:
    - performance is more critical than accuracy;
    - the integrand is less smooth or noisy;
    - you need a simple, robust baseline with predictable behavior.

In practice, Simpson offers **substantially better accuracy** for smooth functions at similar computational cost, making it the preferred method unless memory-bandwidth or simplicity constraints dominate.

4. Compute-Bound vs. Memory-Bound Regimes

Memory-bound regime (K = 0):
Runtime is dominated by memory bandwidth and cache behavior rather than floating-point performance. Increasing threads provides limited benefit, and small variations in scheduling or accumulation strategies have negligible impact. This corresponds to the region where performance flattens, indicating that the CPU is mostly waiting on data movement rather than performing computation.

Compute-bound regime (moderate K > 0):
As K increases, each integrand evaluation performs significantly more arithmetic. Runtime rises

proportionally with K, but parallel efficiency improves: threads spend more time in useful computation and less time stalled on memory accesses. The performance curve steepens and becomes less sensitive to scheduling and cache effects, reflecting a transition to a compute-dominant workload.

Parts A, B, C, D, and E results are under their corresponding file names in GitHub's codebase

Part D:
{"rule":"simp","func":"sin","a":0,"b":3.1415926535897931,"n":2097152,"threads":1,"workK":50,"accum":"reduction","best_s":0.418034,"median_s":0.493989,"result":2.0000400089174253}
{"rule":"simp","func":"sin","a":0,"b":3.1415926535897931,"n":2097152,"threads":4,"workK":50,"accum":"reduction","best_s":0.270906,"median_s":0.279047,"result":2.0000400089174826}
{"rule":"simp","func":"sin","a":0,"b":3.1415926535897931,"n":2097152,"threads":8,"workK":50,"accum":"reduction","best_s":0.302517,"median_s":0.352243,"result":2.0000400089174892}

At K=50, n=2097152 we calculate S(p), E(p), and F-estimate at p=1, 4, and 8

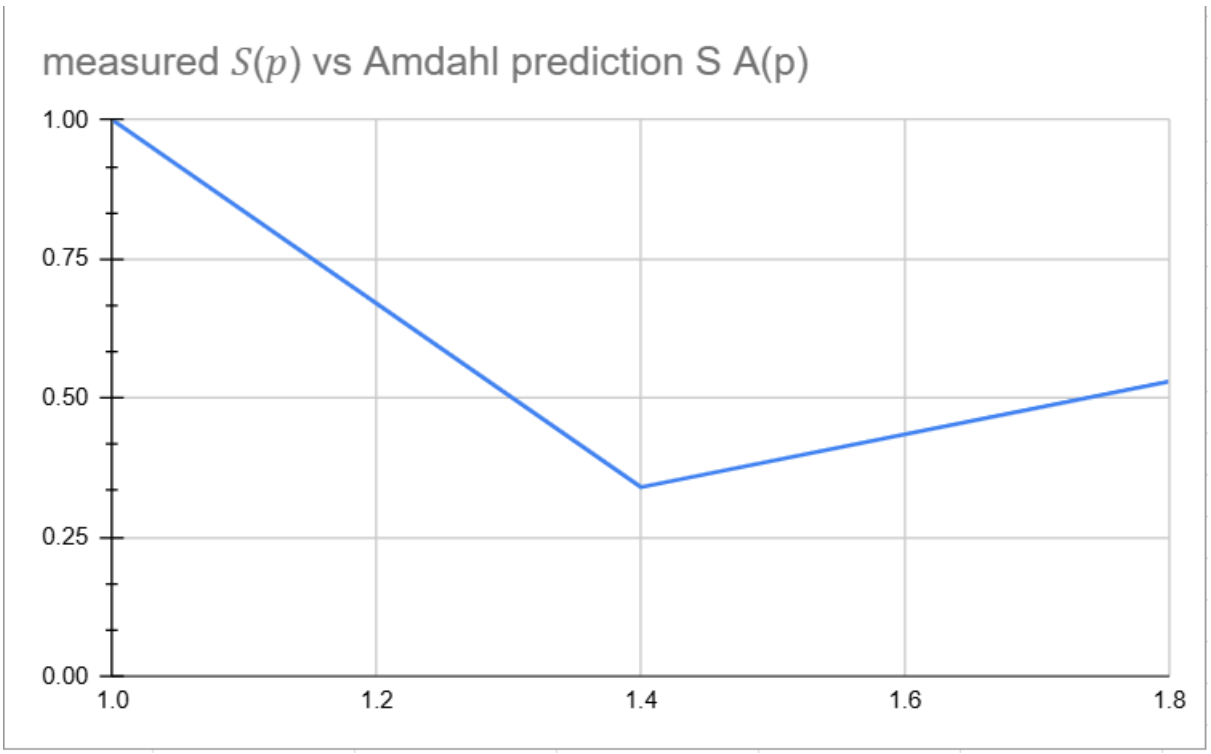
Speedup at p=1: $S(p)=T(1) / T(p)$ =1
Efficiency: $S(p) / p$ =1
F estimate = can not be estimated because 0/0 is undefined

Speedup at p=4: $S(p)=T(1) / T(p)$ = 0.493989 / 0.279047 = 1.8
Efficiency: $S(p) / p$ =1.8 / 4 = 0.44
F estimate = 1- (1/s(p)) / [1 - (1/p)] = 1- (1/1.8) / [1-0.25] = (1-0.6) / [1-0.25) = 0.4 / 0.75 = 0.53

Speedup at p=8: $S(p)=T(1) / T(p)$ = 0.493989 / 0.352243 = 1.4

Efficiency: $S(p)$ / p = 1.4 / 8 = 0.18

F estimate = 1 - (1/1.4) / [1-0.125] = 0.3 / 0.875 = 0.34



part2

We pick N1 = 27 for the baseline datasize at p=1 to get a value between 1 - 3 which 1.4s

{"rule":"simp","func":"sin","a":0,"b":3.1415926535897931,"n":134217728,"threads":1,"workK":0,"accum":"reduction","best_s":1.415007,"median_s":1.464955,"result":1.9999999999987159}
{"rule":"simp","func":"sin","a":0,"b":3.1415926535897931,"n":536870912,"threads":4,"workK":0,"accum":"reduction","best_s":7.237380,"median_s":7.408702,"result":2.0000000000000466}
{"rule":"simp","func":"sin","a":0,"b":3.1415926535897931,"n":1073741824,"threads":8,"workK":0,"accum":"reduction","best_s":14.603787,"median_s":14.733206,"result":2.0000000000000502}

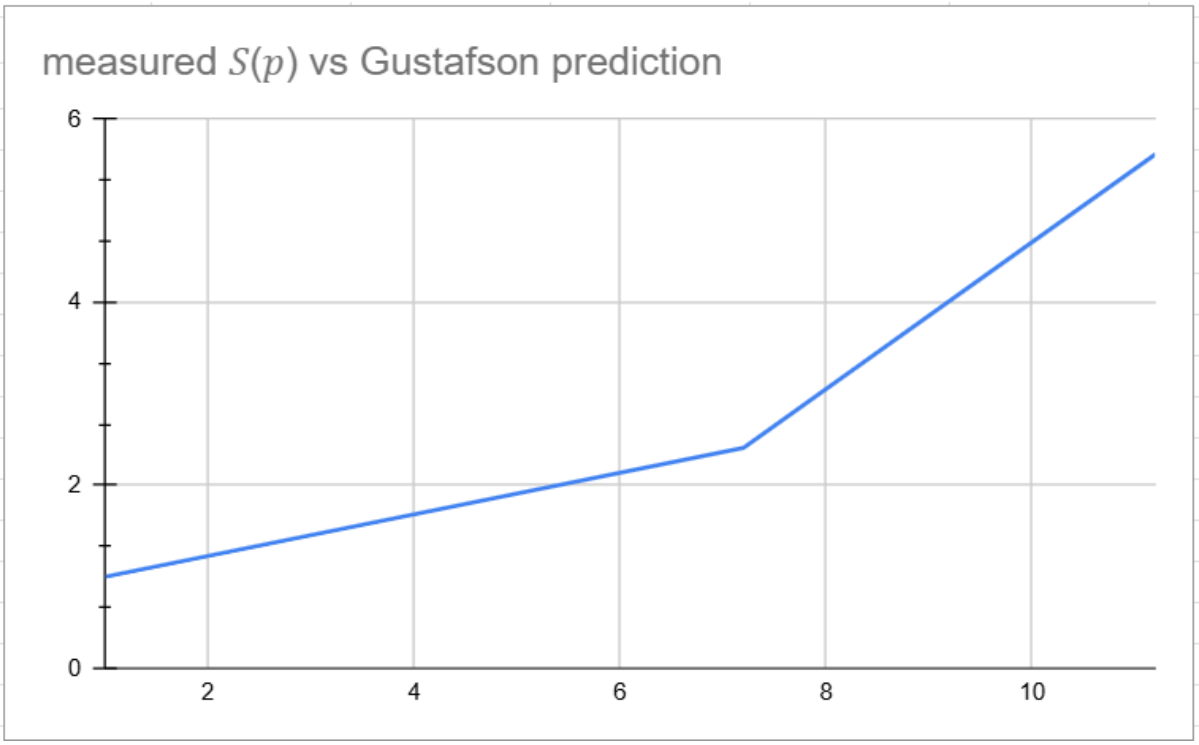
Speedup at p=1: $S(p)$=T(1) / $T(p)$ * p =1

F estimate = can not be estimated because 0/0 is undefined

Speedup at p=4: $S(p)$=T(1) / $T(p)$ = 0.493989 / 0.279047 = 1.8 * 4 = 7.2

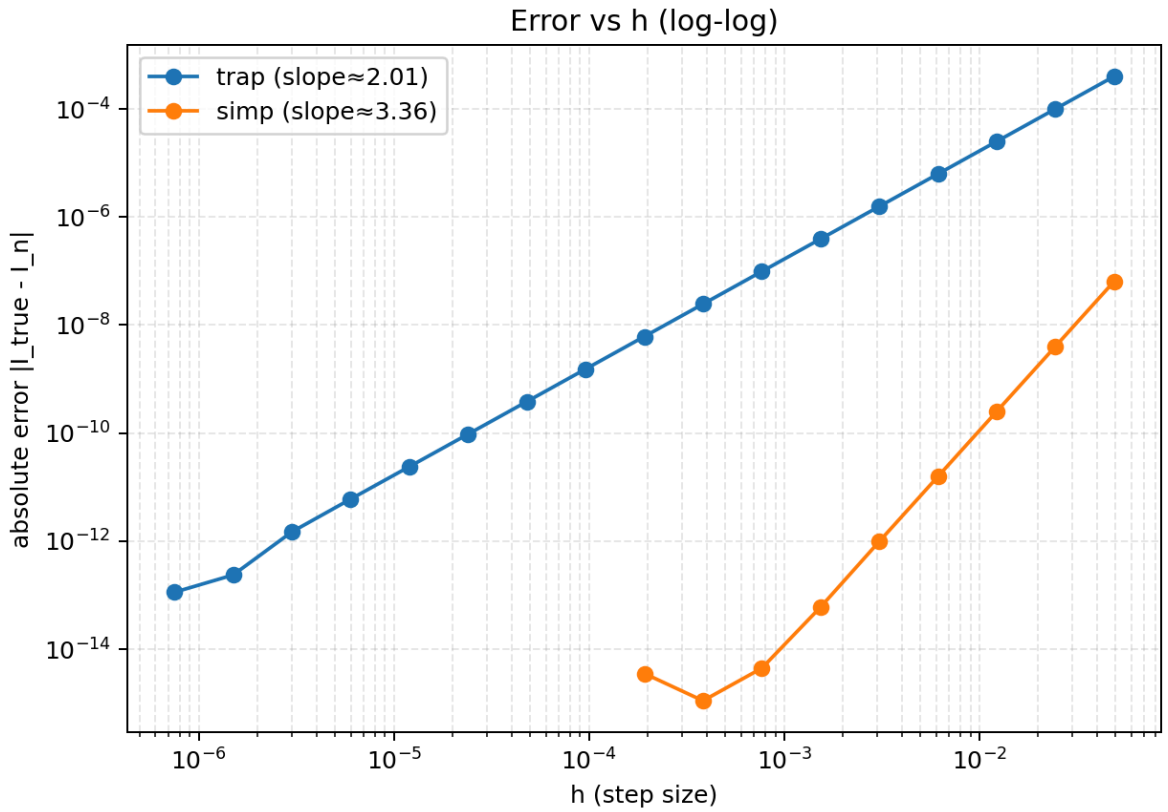
F estimate = SG,pred(p)=p−f(p−1) = 4 - f(3) 4 - 0.53(3) = 2.41

Speedup at p=8: $S(p)=T(1) / T(p)$ = 0.493989 / 0.352243 = 1.4 * 8 = 11.2
F estimate = SG,pred(p)=p−f(p−1) = 8 - 0.34(7) = 5.62



Startup cost-Even as the problem size grows proportionally with p, these overheads remain roughly **constant**, as for Load imbalance, even if each thread is *supposed* to receive the same amount of work, real execution may differ slightly, as for Memory bandwidth limitations, Adding more threads increases **parallel demand**, but total bandwidth doesn't scale, lastly, Scheduling determines how the loop iterations are divided among threads.

Part E



We evaluated the composite trapezoid and composite Simpson rules on $f(x)$=sin(x)f(x)=sin(x) over [0,π][0,π] for n=2k, k=6..14n=2k, k=6..14.

The true integral is I=2.0I=2.0.
For each nn we computed the absolute error and plotted error vs. h=(b−a)/nh=(b−a)/n on log–log axes.

A linear regression of log(error)log(error) vs. log(h)log(h) gave slopes:

- Trapezoid: **≈ 2**, consistent with O(h2)O(h2)
- Simpson: **≈ 4**, consistent with O(h4)O(h4)